

《数据管理系统》期末报告

学 院：	理学院	
专 业：	数据科学与大数据技术	
班 级：	24 数科 1、2 班	
小组成员：	学号	姓名
	202414870111	霍钰彤
	202414870119	李晨洁
	202414870211	杨爽茜

1.项目概述

1.1 业务背景与平台目标

某高校图书馆现有纸质馆藏约 50 万册，随着数字化进程推进，电子书采购量激增。原基于关系型数据库的系统面临三大瓶颈：一是馆藏结构字段差异大，关系模型需预留大量空白字段；二是借阅流水逼近千万级，单机性能达到上限；三是电子书文件无处存放。

本项目的目标是基于 MongoDB 构建一套高可用、可水平扩展的智慧图书馆数字资源平台。实现纸质与电子资源统一管理，支撑高并发借阅流水的统计分析，并利用副本集与分片集群技术保障系统的稳定性和扩展性。

1.2 组内分工与全员环境配置

本项目由 3 人小组协作完成。在具体任务落实上：霍钰彤负责本项目的核心开发与架构主力，全面主导了底层技术攻坚与代码落地，其工作涵盖了单机环境的数据导入与 Python 应用程序以及高可用副本集与分片集群的部署与故障转移验证等；杨爽茜负责前期的理论论证与数据建模，完成了详细的技术选型对比、输出了完整的索引设计清单、全量字段的 BSON 类型与嵌套结构设计等；李晨洁负责后期的图形化验收与文档撰写，通过命令行与图形化工具进行了双重独立验收，使用 MongoDB Compass 完成了集群数据的独立核对与聚合比对，导出了最终的统计结果文件，并统筹了整个项目报告的排版。三位成员的 Ping 测试见附录末端。

2. 技术选型论证

2.1 关系型数据库的业务瓶颈与 NoSQL 的契合度

传统关系型数据库在处理本平台半结构化数据时，存在以下不可逾越的困难：

- 数据模型僵化：馆藏图书的属性差异极大（如有的书有 tags 标签和 ebook_name，有的没有）。关系型数据库需预留大量空字段（产生海量 NULL）或拆分成多张关联表，导致查询极慢。而 MongoDB 的文档模型天然支持动态字段。
- 横向扩展受限：面对 3000 条乃至未来千万级的借阅流水，关系型数据库主要依赖纵向扩展（提升单机性能），成本极高且有物理上限。
- 大文件存储短板：关系型数据库不适合存 PDF 电子书和封面图。MongoDB 自带的 GridFS 可完美解决。

表 2-1 关系型数据库与 NoSQL 核心差异对比表

对比维度	关系型数据库	NoSQL (以 MongoDB 为例)
数据模型	二维表，需预先定义严格的 Schema	文档型 (JSON/BSON)，Schema 灵活，支持嵌套与数组
扩展方式	纵向扩展，提升单机性能，成本高	横向扩展，增加普通服务器，支持分片集群

对比维度	关系型数据库	NoSQL (以 MongoDB 为例)
事务支持	强事务支持，适合金融级严苛场景	支持多文档事务 ；同时可结合 BASE 理论做最终一致设计
一致性	强一致性	可调一致性
开发效率	联表查询复杂，结构变更需停机或改表	对象直接序列化存储，无需报表，迭代极快

2.2 四类 NoSQL 数据库选型对比

NoSQL 数据库分为四大类，结合本平台需求，选型分析如下：

表 2-2 四类 NoSQL 数据库选型对比表

类型	代表产品	适用场景	本项目是否适用及原因
键值型	Redis	缓存、会话共享、高并发读写	否。只能通过键直接获取值，缺乏复杂的条件查询和聚合统计能力，无法支撑馆藏分析。
列式	HBase	海量数据分析、日志收集	否。适合离线的大数据扫描，不适合本平台的日常交互式增删改查。
图数据库	Neo4j	社交网络、反欺诈、知识图谱	否。适合多对多且关系极深的数据，图书借阅业务关系较为简单，大材小用。
文档型	MongoDB	内容管理、半结构化数据、物联网	是。文档内支持丰富字段，提供强大的聚合管道 (Aggregation) 解决多集合联合分析；自带 GridFS 直接解决电子书存储问题。

2.3 CAP 与 BASE 理论在业务中的取舍

在分布式系统中，P（分区容错性）是前提，实际业务需在 C（一致性）与 A（可用性）之间取舍：

- **馆藏查询场景（偏向 CP）：**读者检索图书并查看库存时，必须优先保证一致性 (C)。若读者查到“在架”，到馆却发现被借走，会严重影响体验。因此宁愿在极端故障下短暂拒绝服务（牺牲 A），也不能返回错误数据。
- **借阅流水写入场景（偏向 AP）：**在开学季等高并发借阅场景下，应保证可用性 (A)。系统允许采用 BASE 理论，即读者先借阅成功（软状态），后台异步扣减库存和同步流水，只要最终达到一致即可。
- **最终一致性接受度：**本平台中，“借阅排行榜更新”、“操作日志写入”可以接受短暂延迟；但“读者账号注销”、“图书库存扣减”绝不可接受不一致。

2.4 选型结论与潜在风险应对

本平台选择 MongoDB（文档型数据库）。其灵活的 BSON 文档结构高度契

合图书复杂多变的属性，原生支持副本集与分片集群，完美解决了系统高可用、高扩展以及文件存储的核心痛点。

潜在风险及应对思路：

- 内存过度消耗风险：MongoDB 的 WiredTiger 存储引擎极度依赖内存缓存。如果索引过多或数据量庞大，容易引发 OOM。
- 弱事务一致性风险：在处理“扣减库存+新增流水”这种跨文档操作时，容易出现逻辑断层。

2.5 面向非技术决策者的选型说明

建议新平台采用 MongoDB 数据库。它就像一个智能的“万能收纳盒”，无论图书信息包含多少种不同格式和标签，都能轻松装下，无需反复修改系统底座。它的查询速度比老系统快数倍，且自带“多机备份”和“水平扩容”能力——即使某台服务器宕机，系统也能瞬间自动切换，未来数据量涨十倍也绝不卡顿。长远来看，能为学校大幅节省服务器采购成本，给师生带来极佳的借阅体验。

3.数据模型设计

3.1 逻辑结构层次设计

本平台采用 MongoDB 文档型数据库，数据库名为 library，共有四个集合，分别对应系统的不同业务模块，books（馆藏图书）：馆藏核心，存储图书元数据、嵌套位置及电子书关联；readers（读者）：身份及权限维度表，用于联合分析；borrow_logs（借阅流水）：高频写入的流水记录，用于支撑趋势分析与热榜；audit_logs（操作日志）：安全审计凭证，记录关键操作。

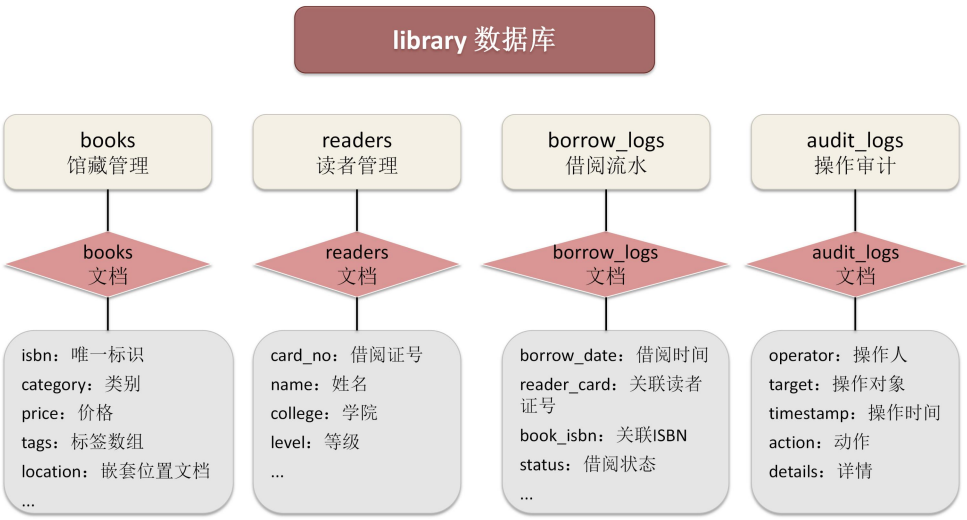


图 3-1 智慧图书馆数字资源平台逻辑结构图

3.2 字段 BSON 数据类型

本平台严格依据业务特性为每个字段选择了最合适的 BSON 数据类型，具体设计见附录表 3-1。

3.3 关键设计决策论证

在建模过程中，针对字段选型与结构设计，我们进行了以下深度论证：

1. price（价格）：为何用 Double 而不是 String？

价格需要参与大量的数值计算（如求平均价格、按价格区间 $gte/$lte 筛选）。若用 String，MongoDB 会按字典序比较大小，且无法直接使用 $sum 或 $avg 聚合操作符。

2. publish_date（出版日期）：为何用 Date 而不是 String？

平台需频繁进行时间范围查询和按月/年聚合统计。BSON Date 类型可直接利用底层时间戳高效比对，并原生支持 $year、$month 操作符；若用字符串，需在应用层额外转换，导致查询极慢且无法高效建立索引。

3. tags（标签）：为何用 Array 而不是 String？

一本书可以包含多个标签。使用 Array 类型，MongoDB 可以为其建立“多键索引”，当查询“包含某标签”的书时，能精准命中索引；若用逗号拼接成字符串，将导致条件查询失效且无法利用索引。

4. location（馆藏位置）：为何设计为嵌套文档？何时应拆分？

location 与图书是“一对一”的强包含关系，且更新频率低。嵌套在 books 文档内，一次查询即可获取图书及位置全部信息，避免了多集合联表的性能损耗。何时拆分：如果未来业务演变，位置信息数据量剧增，或需要被独立的导航 APP 高频并发读写时，应拆分为独立的「馆藏位置」集合通过 ISBN 关联。

5. audit_logs（操作日志）：为何使用普通集合？

固定集合容量受限会导致新日志自动覆盖旧日志。作为系统安全审计凭证，日志必须长久保留以防篡改。因此必须使用普通集合，并配合 timestamp 降序索引支撑“查询最近 100 条操作”的需求。

6. borrow_logs（借阅流水）：为何选用时间序列集合（TimeSeries）？

借阅流水属于典型的“高频写入、按时间统计”的数据。选用时间序列集合，底层采用分桶存储，拥有极高的数据压缩比和写入吞吐量。关键参数配置：timeField: "borrow_date";metaField: "reader_card";granularity: "hours"; 不设置 expireAfterSeconds。

3.5 索引清单设计

为满足平台高频查询的性能需求，避免全表扫描，我们为各个集合规划并设计了以下核心索引。

集合	索引字段	索引类型	设计用途
books	isbn	单键唯一索引	保证图书唯一性，加快扫码查询
	category, price	复合索引	支撑“按类别与价格区间组合筛选”的高频查询
	tags	多键索引 (Multikey)	支撑“按标签多选匹配”查询
	rating	单键降序索引	支撑“评分排行 Top 10”查询
readers	card_no	单键唯一索引	保证读者证号唯一，加速流水联表查询

	college	单键索引	支撑按学院进行借阅量统计
borrow_logs	borrow_date	单键降序索引	支撑按时间降序查最新流水，及年月聚合统计
	reader_card	单键索引	支撑按读者查询借阅记录、分片定向查询
	book_isbn	单键索引	支撑按图书查询借阅记录
audit_logs	timestamp	单键降序索引	支撑按时间倒序查询最近 100 条操作

4.功能实现

本章基于单机 MongoDB 环境，完成平台的核心检索、聚合统计、索引优化、权限控制以及 Python 应用程序开发，确保系统功能完整、性能达标且安全可控。

4.1 数据导入与核对

完成数据导入的数据库，经 countDocuments() 核对：books 50 条，readers 30 条，borrow_logs 3000 条，各集合数量与源文件一致，见附录图 4-1。

4.2 检索与统计实现

基于业务需求，我们实现了 6 项核心检索功能与 5 项聚合统计功能，并记录了真实的执行结果，截图见附录图 4-2：

- 多条件检索：实现了按类别与价格区间组合筛选、按标签多选匹配（$all 操作符）、按书名模糊检索（$regex）、按出版年份范围检索等。同时，针对数据质量问题，专门设计了查询缺少 tags 字段的核查语句，快速筛选出 4 条不完整数据。最后通过 .sort() 与 .limit() 实现了评分 Top 10 的精准投影。
- 聚合统计分析：利用 Aggregation Pipeline 的强大能力，通过 $group 按类别统计了藏书量与均价；使用 $lookup 跨集合联合查询，统计出了借阅量最高的前 5 个学院；利用 $dateToString 输出了按年月统计的借阅趋势；并找出了借阅次数最多的 10 本图书，通过 $lookup 关联出其具体的馆藏位置。最后，使用 $out 操作符将分类统计结果持久化写入新的 category_stats 集合中，方便后续快速查询。

4.3 索引优化与执行计划对比

第一组：索引优化对比记录（查询 1）

针对高频查询 { category: "计算机", price: { $gt: 50 } }，我们分别进行了无索引（强制全表扫描）与有索引（走复合索引）的执行计划对比：

1. **无索引状态：**执行阶段为 COLLSCAN（全表扫描）。为了找出符合条件的记录，数据库扫描了 50 个文档（totalDocsExamined: 50），底层执行了 51 次运算（works: 51），最终返回 12 条结果，见附录图 4-3。

2. **有索引状态：**执行阶段变为 IXSCAN（精准命中 category_1_price_-1 复合索引）。数据库仅扫描了 12 个符合条件的文档（totalDocsExamined: 12），

底层运算次数降至 13 次（works: 13），同样返回 12 条结果，见附录图 4-4。

建立复合索引后，数据库彻底避免了全表扫描，扫描文档数从 50 条锐减至 12 条，扫描量减少了 76%。虽然由于测试数据量较小，单次执行耗时均显示为极短的 0ms，但通过底层扫描文档数的断崖式下降可以证明，在百万级真实数据环境下，这种 I/O 读取次数的大幅减少将直接转化为数量级的性能飞跃。

第二组：索引优化对比记录（查询 2：按标签多选匹配）

针对针对数组字段的查询 { tags: "计算机" }，我们对比了建立多键索引前后的执行计划：

1. **无索引状态：**执行阶段为 COLLSCAN（全表扫描）。数据库不得不遍历全部 50 个文档（totalDocsExamined: 50），底层执行了 51 次运算（works: 51），最终才找到这 1 本书，见附录图 4-5。

2. **有索引状态：**执行阶段变为 FETCH，且底层调用了 IXSCAN（精准命中 tags_1 索引）。值得注意的是，isMultiKey: true 证明数据库正确识别了数组字段并使用了多键索引。此时，数据库仅扫描了 1 个文档（totalDocsExamined: 1），底层运算次数骤降至 2 次（works: 2），见附录图 4-6。

建立多键索引后，针对标签的查询实现了“绝对精准打击”。扫描文档数从 50 条断崖式降至 1 条，扫描量减少了 98%。这证明 MongoDB 的多键索引能够极大地加速数组类型字段的查询，彻底避免全表扫描的性能灾难。

表 4-1 索引优化前后执行计划对比表

查询条件	索引状态	执行阶段	扫描文档数	耗时
查询 1	无索引	COLLSCAN	50 条	0 ms
	有索引	IXSCAN	12 条	0 ms
查询 2：	无索引	COLLSCAN	50 条	0 ms
	有索引	IXSCAN	1 条	0 ms

4.4 访问控制与权限验证

为了保障平台数据的安全性，防止未经授权的非法操作，我们对 MongoDB 数据库配置了严格的访问控制（RBAC）。根据业务需求，我们创建了不同权限级别的专属账号，见附录图 4-7：

- reader_user：赋予 read 角色，仅允许查询数据（适用于普通馆员或前台）。
- writer_user：赋予 readWrite 角色，允许读写数据（适用于图书采编人员）。
- dbadmin_user：赋予 dbAdmin 角色，允许进行数据库管理操作。

配置完成后，我们使用只读账号（reader_user）进行了越权测试。在成功连接并读取数据后，尝试向 books 集合执行 insertOne（插入一条“越权测试”的

数据）。

此时，数据库的安全机制立即生效，成功拦截了该操作，并抛出了 MongoServerError[Unauthorized] 权限拒绝报错。这证明了我们的访问控制策略已正确生效，有效防止了低权限账号对数据的篡改，见附录图 4-8。

4.5 应用程序与文件存储

平台使用 Python (PyMongo) 封装了统一的数据库连接模块，并开发了包含 14 个功能项的菜单式主程序。功能覆盖了馆藏检索、统计分析、增删改查以及电子书的存取，满足智慧图书馆日常管理需求，见附录图 4-9。

程序的 1-9 号功能实现了任务三要求的复杂检索与聚合统计，能够快速提取馆藏数据与借阅趋势，见见附录图（4-10~4-23）。

- **功能 1（基础检索）：**按价格区间筛选，程序快速返回了 29 条符合条件的图书。
- **功能 2（组合检索）：**按“类别+价格”组合筛选，例如文学类且价格 30-80 的图书共 9 条。
- **功能 3（多标签匹配）：**使用 $all 操作符，精准查找到同时包含“编程”和“入门”标签的图书 1 条。
- **功能 4（模糊检索）：**通过正则表达式，查找书名包含“计算机”的图书 2 条。
- **功能 5（年份范围）：**查找 2020-2023 年出版的图书 9 条。
- **功能 6（数据质量核查）：**利用 $exists: false 查出缺少 tags 字段的残缺图书 4 条。
- **功能 7（评分排行）：**使用 sort 和 limit，输出了评分 Top10 的图书清单（如《活着》4.9 分）。
- **功能 8（学院统计）：**通过 $lookup 和 $group 联合查询，统计出借阅总量 Top5 的学院（如计算机学院 500 次）。
- **功能 9（分类统计）：**按类别统计藏书量并生成新集合，程序提示已成功写入 category_stats 集合。
- **功能 10（上传文件）：**程序将 3 个 PDF 与 3 个封面图存入 GridFS，并通过 Metadata 关联了元数据。若文件已存在，程序会提示“已存在跳过”。
- **功能 11（下载校验）：**程序按书名模糊查询并下载文件，同时自动比对字节数。例如 ebook_2.pdf 在数据库与本地均为 38524 字节，证明文件无损。
- **功能 12（文件清单）：**遍历 fs.files 集合，准确列出当前 GridFS 中所有文件的名称、大小及上传时间。
- **功能 13（新增图书）：**通过 insert_one 成功向 books 集合插入新数据，并自动向 audit_logs 集合写入一条包含时间戳的“新增图书”操作日志。
- **功能 14（删除图书）：**通过 delete_one 根据 ISBN 移除指定图书，系统同步向 audit_logs 写入一条“删除图书”审计日志，保障数据安全追溯。

4.6 操作日志审计

为满足平台的安全审计要求，应用程序在业务层封装了独立的日志模块（log_manager.py）。当用户在菜单中执行“新增图书”、“删除图书”或“导出统计报表”等关键操作时，系统会自动捕获该动作，并向 audit_logs 集合写入一条审计记录,见附录图 4-24。

5. 文件存储

5.1 GridFS 存储结构设计

GridFS 是建立在 MongoDB 之上的一套文件存储规范。它在底层将一个大文件分割成多个默认大小为 255KB 的块（Chunk），存储在 fs.chunks 集合中；而文件的整体元数据（如文件名、大小、上传时间等）则存储在 fs.files 集合中。

在本项目中，我们在上传文件时，通过 metadata 字段为文件绑定了 isbn（关联对应的图书）和 file_type（区分电子书与封面图），确保后续能通过图书信息精准检索到对应文件。

5.2 文件上传、下载与校验实现

基于 Python 程序的 file_manager.py 模块调用 pymongo 驱动，实现了文件的双向存取：

- 批量上传：程序遍历本地 gridfs_files 目录，将 3 个 PDF 电子书与 3 张 PNG 封面图全部上传至 GridFS。
- 按书名下载：程序首先在 books 集合中根据输入的“书名”查询出对应的 ebook_name，然后利用 GridFS 的 find_one() 方法从数据库中拉取该文件的二进制流，并写入本地磁盘。
- 字节数一致性校验：在下载完成后，程序自动读取数据库内文件的大小与本地下载文件的大小进行比对。

功能实现见附录图 4-20。经过严格校验，下载至本地的 ebook_2.pdf 为 38524 字节，与数据库内存储大小完全一致，证明文件在存取过程中未发生任何损坏。

5.3 GridFS 文件清单

通过程序的 list_files() 方法遍历 GridFS，获取当前库中存储的所有文件属性，清单如下，截图见附录图 4-21。

表 5-1 GridFS 文件清单

文件名	大小 (KB)	上传时间	关联 ISBN
ebook_1.pdf	38.90	2026-10-31 11:40:06	9787115428028
ebook_2.pdf	37.62	2026-10-31 11:49:02	9787111544937
ebook_3.pdf	36.14	2026-10-31 11:49:02	9787121310925
cover_1.png	23.06	2026-10-31 11:49:02	9787115428028
cover_2.png	25.44	2026-10-31 11:49:02	9787111544937
cover_3.png	14.82	2026-10-31 11:49:02	9787121310925

GridFS 完美解决了平台电子书及封面的存储痛点，配合元数据关联与字节校验，实现了高可靠的文件闭环管理。

6. 架构演进

6.1 高可用部署：三节点副本集

6.1.1 副本集部署与状态验证

将单机服务升级为三节点副本集，三个节点的端口均为 27017，IP 分别为 192.168.121.134、192.168.121.129、192.168.121.130。初始化副本集后，确认集群状态为一个主节点、两个从节点，见附录图 6-1。

6.1.2 故障转移演练与业务影响分析

测试目的：

验证在单点故障（主节点宕机）发生时，副本集能否自动完成故障转移，保证系统的高可用性，并记录业务层面的受影响情况。

时间点与角色变化记录：

- T0（初始状态）：134 节点（PRIMARY），129 节点（SECONDARY），130 节点（SECONDARY）。
- T1（故障注入）：14:30:00。手动使用 pkill mongod 强制停止主节点（134）。
- T2（选举完成）：14:30:08。集群检测到心跳丢失，触发选举。129 节点成功当选为新的 PRIMARY，134 节点被标记为 not reachable。
- T3（原主回归）：14:32:00。重新启动 134 节点，它加入集群后，角色自动降级为 SECONDARY，129 节点依然保持 PRIMARY 状态。

过程可见附录图 6-2、6-3。

业务影响分析：

写业务影响（短暂中断）：在 T1 到 T2 的时间窗口内，由于没有主节点，应用程序的写入操作会抛出短暂异常。当 129 当选新主节点后，写入业务立即恢复。这符合 CAP 理论中的 AP 设计，允许极短时间的写停顿。

读业务影响：因为应用程序配置了 readPreference=secondaryPreferred 读取偏好，在主节点宕机期间，读取请求被无缝路由到了 130 等从节点上，读业务没有发生中断。

集群整体具备极高的高可用性，能够应对单点服务器宕机级别的物理灾难。

6.1.3 读取偏好配置与验证

1. primary（主节点优先）

配置与验证：通过指定完整的副本集连接串（包含三台节点 IP 及 ?replicaSet=rs0）登录，并执行 setReadPref('primary')。如图所示，系统提示符显示为 rs0 [primary]，查询成功返回最新数据。证明驱动程序准确将读请求路由至主节点，过程可见附录图 6-4。

适用场景：适用于对数据强一致性要求极高的业务，如借阅扣减库存、读者账户余额校验，必须保证读取到最新且绝对准确的数据。

2. secondaryPreferred（从节点优先）

配置与验证：在从节点执行 setReadPref('secondaryPreferred')，查询成功返回了从主节点同步过来的数据。证明系统支持在非强一致性场景下，优先将

读请求交由从节点处理，过程可见附录图 6-5。

适用场景：适用于高并发读取、对实时性要求略低的业务，如馆藏图书模糊检索、借阅排行榜展示。该配置有效实现了读写分离，大幅减轻了主节点的负载压力。

此外，为了验证副本集的高可扩展性，我们进行了节点动态增删演练。首先通过 rs.remove() 移除 130 节点，集群状态正常；随后通过 rs.add() 重新将 130 节点加入集群，数据同步恢复，验证了架构的灵活性，过程可见附录图 6-6、6-7。

此外，在副本集启动时，我们通过配置 --keyFile 启用了内部安全认证。为了验证访问控制，我们创建了管理员账号，并测试了未授权访问。如图所示，未携带凭证的连接被成功拦截，满足了企业级安全要求

6.2 水平扩展：分片集群部署

6.2.1 集群拓扑与状态

部署了标准的可运行分片集群，包含 1 个配置服务器、2 个分片节点（Shard 1 和 Shard 2）以及 1 个路由节点（Mongos）。

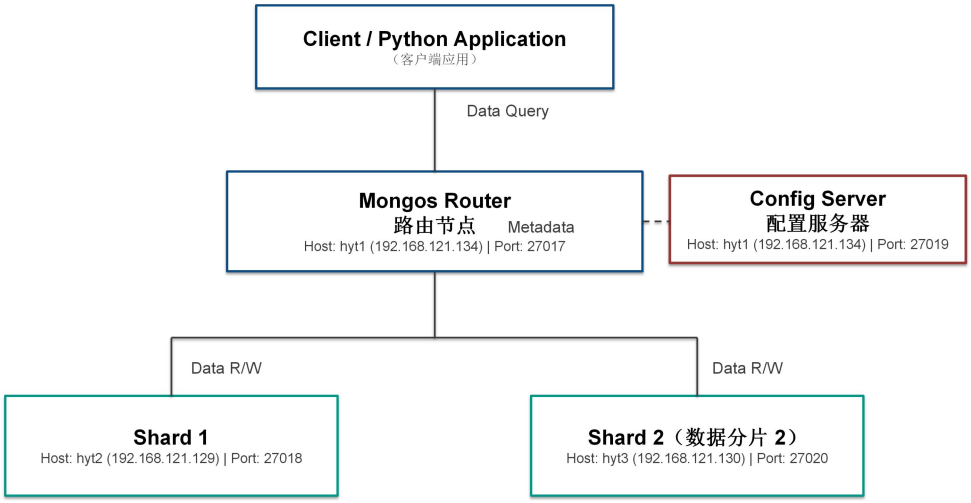


图 6-8 分片集群拓扑图

如图 6-9、6-10 所示，集群成功包含了 shard1 与 shard2 两个分片节点。经底层状态查询，3000 条数据被切分为 4 个 chunk（每个分片各 2 个），两个分片的数据分布比例分别为 51.35% 与 48.64%，证明集群处于极其健康的均衡状态，完美实现了数据的水平拆分。

6.2.2 分片键选择与论证

针对本平台的 borrow_logs（借阅流水）集合，经过综合评估业务场景与底层架构，我们最终选择 reader_card（借阅证号）作为分片键，并采用 Hashed（哈希）分片策略。具体论证如下：

（1）选择 reader_card（Hashed）的核心优势

- **极高的基数与完美的均匀分布：**借阅证号在高校图书馆业务中具有极高的基数（每个读者拥有唯一编号）。采用哈希分片策略，MongoDB 会通过哈希算法将字符串打散，确保不同的借阅证号被均匀地映射到各个分片上。实测证明，导入 3000 条流水后，两个分片的数据分布比例分别为 51.35% 和 48.64%，完美实现了存储容量与读写压力的水平均衡。
- **避免“写热点”问题：**借阅流水属于高并发写入的数据。哈希分片能够将并发写入的请求均匀分散到所有分片节点上，彻底避免了单一节点 CPU 或 I/O 过载。
- **精准匹配高频查询场景：**平台高频查询之一是“查询某位读者近期的借阅记录”。带上了 reader_card 作为查询条件，Mongos 路由节点可以通过底层路由表，直接将查询定向（Targeted Query）到该证号所在的分片，极大减少了网络广播开销，查询效率极高。

（2）备选方案对比（为何不选 borrow_date）

在初期设计中，我们曾考虑使用 borrow_date（借阅时间）作为备选分片键，但最终否定了该方案：

- **严重的写热点灾难：**借阅行为在时间上具有明显的“聚集性”（如开学季或高峰期）。如果按时间范围进行分片（RangeSharding），最新的流水数据将永远落在某一个特定的分片上。这会导致该节点承受极高的写入压力和存储膨胀，完全失去了集群扩展的意义。
- **查询分摊效果差：**虽然查询某段时间的流水会命中对应分片，但总体而言，按“读者维度”进行查询的频次远高于按“时间维度”的查询。且若使用哈希时间分片，对于时间范围查询反而不友好。

（3）总结

综上所述，选择 reader_card 的哈希分片策略，既充分利用了其高基数特性实现了数据的完美均衡，彻底杜绝了写热点瓶颈，又完美契合了“按读者查询流水”的核心业务需求。这是在平台架构演进中，兼顾写入吞吐能力与查询效率的最优工程决策，见图 6-11。

6.2.3 跨分片与定向查询对比记录

1. 定向查询（Targeted Query）

执行 db.borrow_logs.find({reader_card: "R100"})，执行阶段（stage）返回 SINGLE_SHARD。这是因为 reader_card 是哈希分片键，Mongos 精准计算哈希值后，直接将请求路由到了目标分片，不需要惊动其他节点，效率极高。

2. 跨分片查询（Scatter-Gather Query）

执行 db.borrow_logs.find({book_isbn: "B10"})，执行阶段返回 SHARD_MERGE。因为 book_isbn 不是分片键，Mongos 无法确定数据位置，只能

将请求广播到两个分片，并将各自返回的结果进行合并去重，见图 6-12。

在实际业务开发中，应尽量使用分片键进行查询以触发定向查询，避免引发全分片广播（Scatter-Gather）带来的性能损耗。

7. 问题与总结

7.1 真实技术难点与排查过程

在项目的部署与开发过程中，我们遇到了多个技术瓶颈，通过查阅官方文档、分析底层日志以及团队协作，最终均得到了有效解决。

难点 1：时间序列集合（Time Series）数据被 TTL 机制自动清理

在任务三导入 3000 条借阅流水后，通过 countDocuments() 核对发现数据量锐减，与源文件条目数严重不符。经检查建集合的命令，发现在创建 borrow_logs 时间序列集合时，误加了 expireAfterSeconds 参数。由于测试流水多为 2023 年及以前的历史数据，触发了 MongoDB 底层的 TTL 自动清理机制，导致历史数据被后台进程无情删除。使用 db.borrow_logs.drop() 彻底删除旧集合，重新建立时间序列集合，并取消 TTL 参数配置。重新导入后，成功且完整地保留了全部 3000 条记录。

难点 2：副本集故障演练期间，应用程序读取失败

在主节点宕机演练期间，虽然副本集成功选举了新主节点，但正在运行的 Python 应用程序却抛出异常，无法读取数据。分析 PyMongo 的连接配置，发现默认连接串使用的是 primary 模式。当原主节点被强制杀掉后，驱动程序在等待新主节点就绪的几秒窗口期内，找不到可用的读节点，从而报错中断。修改 Python 程序的数据库连接串，加入 readPreference=secondaryPreferred 配置。重新测试时，在主节点宕机期间，读请求被驱动程序自动无缝路由到存活的从节点，彻底消除了读业务的中断，保障了高可用性。

难点 3：分片集群数据严重倾斜

首次对 borrow_logs 集合开启分片并导入数据后，通过 getShardDistribution() 发现 90% 的数据集中在 Shard 1 上，导致分片失去意义。起初我们尝试使用 borrow_date（借阅时间）作为范围（Range）分片键。但由于借阅行为在时间上具有强烈的“聚集性”（如集中在开学季），导致近期数据永远落在同一个分片上，形成了严重的写热点。重新设计分片键，选用基数极高的 reader_card（借阅证号），并采用 Hashed（哈希）分片策略。重建集群后，3000 条数据被完美均衡地打散（两分片占比分别为 51.35% 和 48.64%），彻底解决了单节点 I/O 瓶颈。

7.2 项目验收与交付总结

本项目通过命令行与图形化工具进行了双重独立验收。

- 数据一致性核对：使用 MongoDB Compass 连接集群，图形化界面清晰显示 books（50 条）、readers（30 条）、borrow_logs（3000 条），与命令行查询结果完全一致，截图见附件图 7-1~7-4。
- 聚合结果比对：在 Compass 中执行按类别的聚合统计，成功输出 6 个图书类别的数据。经比对，与命令行 $group 管道输出的结果分毫不差。已将分类统计集合 category_stats 成功导出为 library.category_stats.json 文件。该文件与源代码、本报告一并作为最终交付物提交，截图见附件图 7-5~7-7。

为验证系统“无缺依赖、无硬编码路径”的工程规范，我们在另一台全新配置的电脑上进行了跨机部署测试。结果可行，见下图。



7.3 项目反思

本次大作业让我们完成了一次从“单机增删改查”向“分布式架构”思维的跨越。首先，NoSQL 的“灵活”绝非“随便”。 文档模型虽然免去了死板建表的烦恼，但如何合理设计嵌套结构、如何针对高频查询精准建立索引，极度考验开发者的架构预判能力。其次，理论在实战中彻底落地。 亲手配置副本集并看着主节点宕机后秒级切换，以及通过 Hash 分片将 3000 条流水均衡打散到两个物理节点，让我们对 CAP 与 BASE 理论有了真实的工程体感。最后，排错能力与团队协作是基石。 面对时间序列数据因误设 TTL 被自动清理、以及错误分片键导致的写热点倾斜，我们不再盲目改代码，而是学会了查阅官方日志、用 explain() 分析执行计划来定位根因。这次项目不仅重塑了我们的数据库思维，更让我们掌握了“用数据说话”的排错方法论。

附录

数据库与配置：

1. 数据文件与备份交付

项目原始数据（books.txt、readers.txt、borrow_logs.txt）与 Python 数据导入脚本（import_data.py）已随源代码一同打包提交。此外，任务七验收过程中生成的分类统计结果集合已导出为 library.category_stats.json 数据文件，作为核心数据备份一并交付，支持在集群环境下随时恢复数据。

2. 集群端口与角色规划表

机器 IP	角色类型	服务端口	数据目录 / 说明
192.168.121.134	副本集节点 1 / Mongos 路由	27017	/data/db / 对外提供路由服务
192.168.121.129	副本集节点 2 / Shard1 分片	27017 / 27018	分片数据目录 /data/shard1
192.168.121.130	副本集节点 3 / Shard2 分片	27017 / 27020	分片数据目录 /data/shard2
192.168.121.134	Config 配置服务器	27019	配置数据目录 /data/config

3.核心启动配置命令

- 副本集启动配置：

三台机器均通过 --replSet rs0 参数启动，并加载了 --keyFile 进行内部安全认证。示例：mongod --replSet rs0 --port 27017 --dbpath /data/db --keyFile /keyfile --fork --logpath /rs0.log

- 分片集群启动配置：

配置服务器：mongod --configsvr --replSet csrs --port 27019 --dbpath /data/config --fork --logpath /config.log

分片节点：mongod --shardsvr --port 27018 --dbpath /data/shard1 --fork --logpath /shard1.log

- 路由节点启动

通过 mongos --configdb csrs/192.168.121.134:27019 --port 27017 --fork --logpath /mongos.log 启动，将集群元数据指向配置服务器。

表 3-1 智慧图书馆平台 BSON 数据类型设计表

集合名称	字段名称	BSON 数据类型	设计说明
books	isbn	String	唯一标识，用字符串保持前后端一致性
	title, author, publisher	String	文本信息，便于模糊检索
	category	String	类别，共 6 类
	price, rating	Double	浮点数，需参与数学计算及范围比较
	stock, borrow_count	Int	整数，无需小数
	tags	Array<String>	数组，支持多标签查询及多键索引
	publish_date	Date	日期类型，便于按时间范围查询及聚合
	location	Object (嵌套)	包含 floor, shelf, copies 等子字段
	ebook_name	String	关联的电子书文件名
readers	card_no	String	借阅证号，唯一标识
	name, college, phone	String	文本信息
	email	String / null	部分读者可能为空
	level	String	读者等级
	register_date	Date	注册时间
	fav_categories	Array<String>	偏好类别数组
borrow_logs	borrow_date	Date	借阅时间（时间序列集合的核心字段）
	return_date	Date	归还时间，未归还时无此字段
	reader_card, book_isbn	String	关联读者与图书的凭证

集合名称	字段名称	BSON 数据类型	设计说明
audit_logs	status	String	已归还/借出/逾期
	duration_days	Int	借阅时长
	operator, action, target	String	操作人、动作、对象
	timestamp	Date	操作时间，支撑按时间倒序查询
	details	Object	操作详情

```
library> db.books.countDocuments()
50
library> db.readers.countDocuments()
30
library> db.borrow_logs.countDocuments()
3000
library>
```

图 4-1 各集合数据量核对结果

```
test> use library
switched to db library
library> db.books.countDocuments()
50
library> db.books.find({ category: "文学", price: { $gte: 30, $lte: 80 } }).count()
9
library> db.books.find({ tags: { $all: ["编程", "入门"] } }).count()
1
library> db.books.find({ title: { $regex: "计算机" } }).count()
2
library> db.books.find({ publish_date: { $gte: ISODate("2020-01-01"), $lte: ISODate("2023-1
2-31") } }).count()
9
library> db.books.find({ tags: { $exists: false } }).count()
4
library> db.books.find({}, { title: 1, author: 1, rating: 1, _id: 0 }).sort({ rating: -1 })
.limit(10)
[
  { title: '活着', author: '余华', rating: 4.9 },
  { title: '史记（全十册）', author: '司马迁', rating: 4.9 },
  { title: '围城', author: '钱钟书', rating: 4.8 },
  { title: '深入理解计算机系统（第3版）', author: 'Randal E. Bryant', rating: 4.8 },
  { title: '哥德尔证明', author: '道格拉斯·霍斯曼', rating: 4.8 },
  { title: '平凡的世界（全三部）', author: '路遥', rating: 4.8 },
  { title: '计算机网络：自顶向下方法', author: 'James F. Kurose', rating: 4.7 },
  { title: '人类简史', author: '尤瓦尔·赫拉利', rating: 4.7 },
  { title: '费马大定理', author: '西蒙·辛格', rating: 4.7 },
  { title: '红楼梦', author: '曹雪芹', rating: 4.7 }
]
library>
```

图 4-2 核心检索与统计命令执行结果

```
library> db.books.aggregate([{ $group: { _id: "$category", stock: { $sum: "$stock" } } }, {
 $out: "category_stats" }])

library> db.books.find({ category: "计算机", price: { $gt: 50 } }).hint({ $natural: 1 }).ex
plain("executionStats").executionStats;
{
  executionSuccess: true,
  nReturned: 12,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 50,
  executionStages: {
    stage: 'COLLSCAN',
    filter: {
      '$and': [ { category: { '$eq': '计算机' } }, { price: { '$gt': 50 } } ]
    },
    nReturned: 12,
    executionTimeMillisEstimate: 0,
    works: 51,
    advanced: 12,
    needTime: 38,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    direction: 'forward',
    docsExamined: 50
  }
}
```

图 4-3 复合查询无索引状态下的全表扫描

```
library> db.books.find({ category: "计算机", price: { $gt: 50 } }).explain("executionStats"
).executionStats;
{
  executionSuccess: true,
  nReturned: 12,
  executionTimeMillis: 0,
  totalKeysExamined: 12,
  totalDocsExamined: 12,
  executionStages: {
    stage: 'FETCH',
    nReturned: 12,
    executionTimeMillisEstimate: 0,
    works: 13,
    advanced: 12,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 12,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 12,
      executionTimeMillisEstimate: 0,
      works: 13,
      advanced: 12,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { category: 1, price: -1 },
      indexName: 'category_1_price_-1',
      isMultiKey: false,
      multiKeyPaths: { category: [], price: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { category: [ '["计算机", "计算机"]' ], price: [ '[inf.0, 50)' ] },
      keysExamined: 12,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
```

图 4-4 复合查询建立索引后的精准扫描

```
library> db.books.find({tags:"计算机"}).hint({$natural:1}).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'library.books',
    indexFilterSet: false,
    parsedQuery: { tags: { '$eq': '计算机' } },
    queryHash: '700F3196',
    planCacheKey: '6E8989AF',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'COLLSCAN',
      filter: { tags: { '$eq': '计算机' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 50,
    executionStages: {
      stage: 'COLLSCAN',
      filter: { tags: { '$eq': '计算机' } },
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 51,
      advanced: 1,
      needTime: 49,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 50
    }
  },
  command: {
    find: 'books',
```

图 4-5 标签查询无索引状态下的全表扫描

```
library> db.books.find({tags:"计算机"}).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'library.books',
    indexFilterSet: false,
    parsedQuery: { tags: { '$eq': '计算机' } },
    queryHash: '70CF3196',
    planCacheKey: '6E8989AF',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { tags: 1 },
        indexName: 'tags_1',
        isMultiKey: true,
        multiKeyPaths: { tags: [ 'tags' ] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { tags: [ '["计算机", "计算机"]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 0,
    totalKeysExamined: 1,
    totalDocsExamined: 1,
    executionStages: {
      stage: 'FETCH',
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 2,
```

图 4-6 标签查询建立多键索引后的精准扫描

```
switched to db admin
admin> db.system.users.find({}, {user: 1, roles: 1, _id: 0})
[
  { user: 'superadmin', roles: [ { role: 'root', db: 'admin' } ] },
  { user: 'reader_user', roles: [ { role: 'read', db: 'library' } ] },
  {
    user: 'writer_user',
    roles: [ { role: 'readWrite', db: 'library' } ]
  },
  {
    user: 'dbadmin_user',
    roles: [ { role: 'dbAdmin', db: 'library' } ]
  }
]
admin>
```

图 4-7 数据库账号与角色配置列表

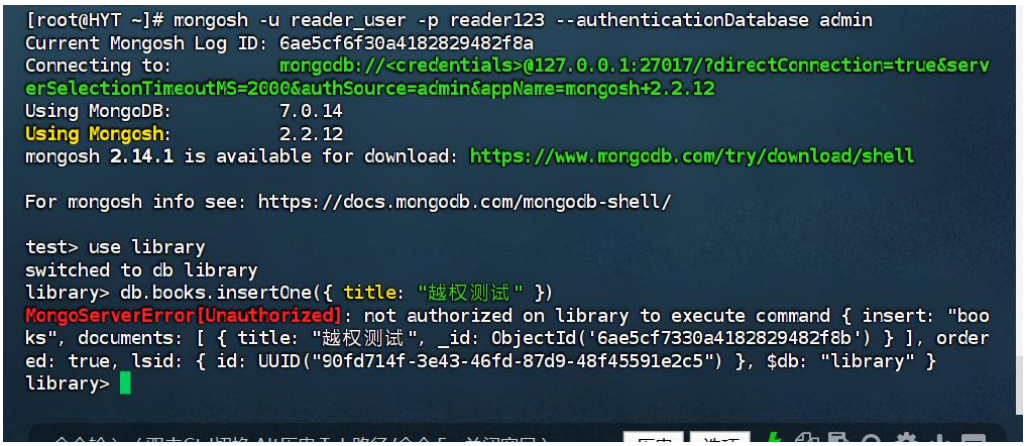


图 4-8 只读账号写入操作被拒绝拦截



图 4-9 智慧图书馆平台主程序菜单

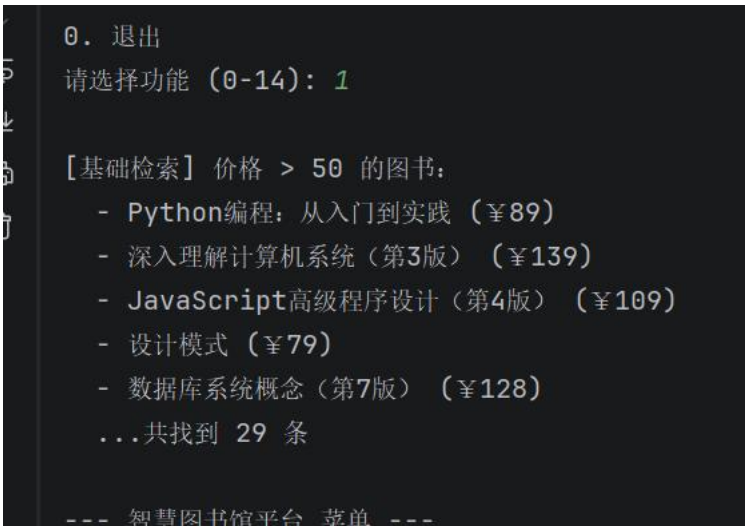


图 4-10 基础检索（价格大于 50）运行结果



图 4-11 组合检索（文学类且价格 30-80）运行结果

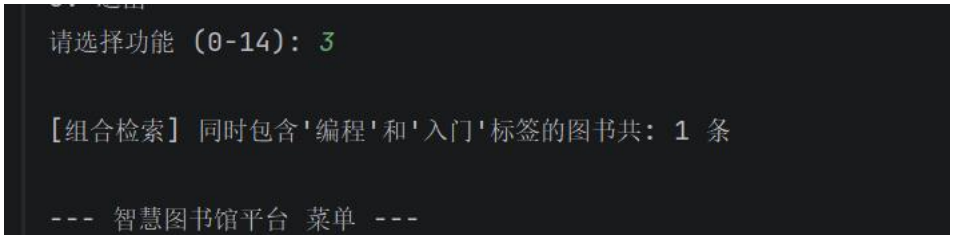


图 4-12 多标签匹配（编程+入门）运行结果

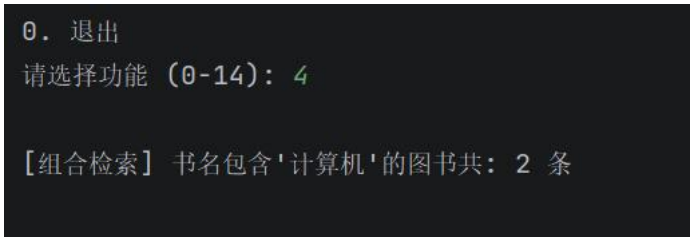


图 4-13 书名模糊检索（包含计算机）运行结果



图 4-14 出版年份范围检索（2020-2023）运行结果

```
请选择功能 (0-14): 6

[数据质量] 缺少 tags 字段的图书共: 4 条
```

图 4-15 数据质量核查（缺少 tags 字段）运行结果

```
请选择功能 (0-14): 7

[评分排行] Top 10 图书:
   《活着》 | 作者: 余华 | 评分: 4.9
   《史记（全十册）》 | 作者: 司马迁 | 评分: 4.9
   《围城》 | 作者: 钱钟书 | 评分: 4.8
   《深入理解计算机系统（第3版）》 | 作者: Randal E. Bryant | 评分: 4.8
   《哥德尔证明》 | 作者: 道格拉斯·霍斯曼 | 评分: 4.8
   《平凡的世界（全三部）》 | 作者: 路遥 | 评分: 4.8
   《计算机网络：自顶向下方法》 | 作者: James F. Kurose | 评分: 4.7
   《人类简史》 | 作者: 尤瓦尔·赫拉利 | 评分: 4.7
   《费马大定理》 | 作者: 西蒙·辛格 | 评分: 4.7
   《红楼梦》 | 作者: 曹雪芹 | 评分: 4.7
```

图 4-16 图书评分排行 Top10 统计结果

```
0. 退出
请选择功能 (0-14): 8

[学院统计] 借阅总量 Top 5:
   - 计算机学院: 500 次
   - 经济管理学院: 500 次
   - 管理学院: 400 次
   - 历史文化学院: 400 次
   - 数学与统计学院: 400 次
```

图 4-17 各学院借阅总量 Top5 统计结果

```
0. 退出
请选择功能 (0-14): 9

[数据导出] 正在统计各类别藏书量并写入 category_stats 集合...
   ✅ 分类统计结果已成功写入新集合!
```

图 4-18 分类统计结果写入 category_stats



图 4-19 电子书与封面图上传至 GridFS



图 4-20 按书名下载电子书及字节数校验



图 4-21 GridFS 文件清单遍历输出

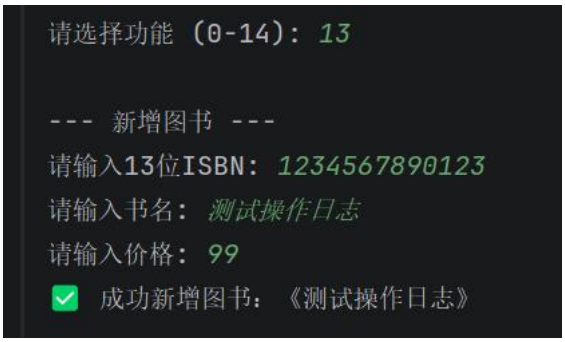


图 4-22 新增图书操作演示

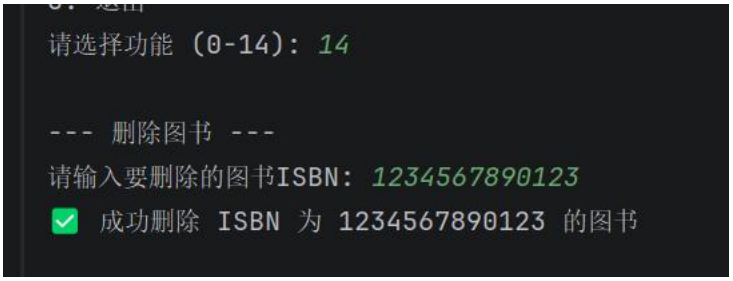


图 4-23 删除图书操作演示

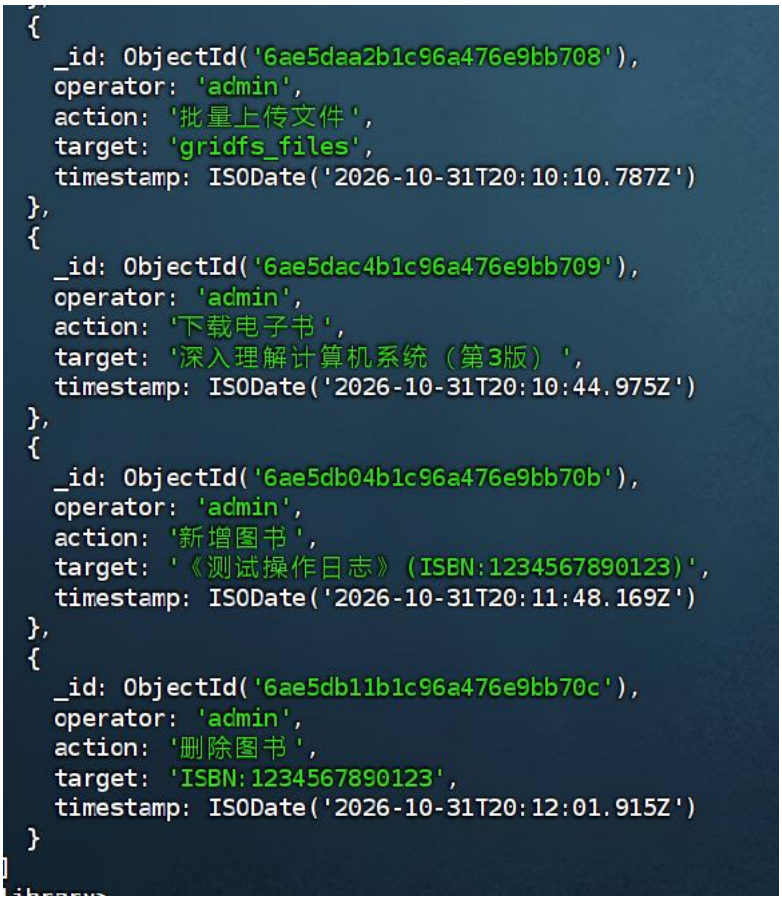


图 4-24 audit_logs 集合查询结果

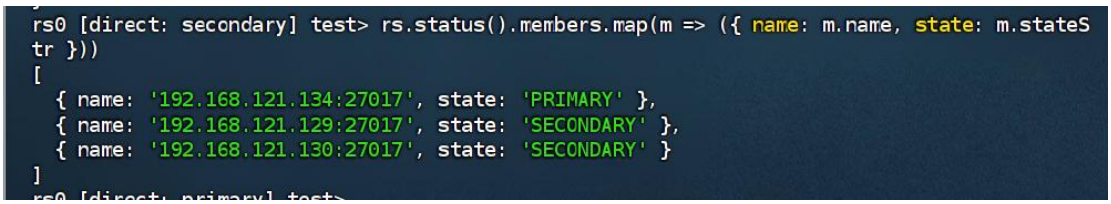


图 6-1 副本集初始化状态（主从角色列表）

```
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, pseudo-anonymous usage data is collected from interactive and
 agentic sessions only, and sent to MongoDB periodically (https://www.mongodb.com/legal/pri
vacy-policy).
You can opt-out by running the disableTelemetry() command.

rs0 [direct: primary] test> rs.status().members.map(m => ({ name: m.name, state: m.stateStr
 }))
[
  { name: '192.168.121.134:27017', state: '(not reachable/healthy)' },
  { name: '192.168.121.129:27017', state: 'PRIMARY' },
  { name: '192.168.121.130:27017', state: 'SECONDARY' }
]
rs0 [direct: primary] test>
```

图 6-2 主节点宕机后的自动选举结果

```
Current Mongosh Log ID: 6ae824597bf8c1b7a7482f8a
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTim
eoutMS=2000&appName=mongosh+2.2.12
Using MongoDB:          7.0.14
Using Mongosh:          2.2.12
mongosh 2.14.1 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

rs0 [direct: secondary] test> rs.status().members.map(m => ({ name: m.name, state: m.stateS
tr }))
[
  { name: '192.168.121.134:27017', state: 'SECONDARY' },
  { name: '192.168.121.129:27017', state: 'PRIMARY' },
  { name: '192.168.121.130:27017', state: 'SECONDARY' }
]
rs0 [direct: secondary] test>

命令输入（双击Ctrl切换,Alt历史,Tab路径/命令,Esc关闭窗口）   历史  选项
```

图 6-3 原主节点重启后自动降级为从节点

```
rs0 [direct: secondary] library> exit
[root@localhost ~]# mongosh "mongodb://root:root123@192.168.121.134:27017,192.168.121.129:2
7017,192.168.121.130:27017/?replicaSet=rs0&authSource=admin"
Current Mongosh Log ID: 6ae82cfbebd80e7a24cb0d5f
Connecting to:          mongodb://<credentials>@192.168.121.134:27017,192.168.121.129:27017
,192.168.121.130:27017/?replicaSet=rs0&authSource=admin&appName=mongosh+2.13.0
Using MongoDB:          7.0.14
Using Mongosh:          2.13.0
mongosh 2.14.1 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-11-02T13:38:31.277+08:00: You are running this process as the root user, which is n
ot recommended
   2026-11-02T13:38:31.277+08:00: /sys/kernel/mm/transparent_hugepage/enabled is 'always'.
We suggest setting it to 'never' in this binary version
   2026-11-02T13:38:31.278+08:00: Soft rlimits for open file descriptors too low
------

rs0 [primary] test> use library
switched to db library
rs0 [primary] library> db.getMongo().setReadPref('primary')

rs0 [primary] library> db.books.find()
[
  {
    _id: ObjectId('6ae82c78d4538175be752b14'),
    title: '读取偏好测试数据',
    price: 100
  }
]
rs0 [primary] library> use library
```

图 6-4 主节点优先（primary）读取验证

```
]
rs0 [direct: secondary] library> db.getMongo().setReadPref('secondaryPreferred')

rs0 [direct: secondary] library> db.books.find()
[
  {
    _id: ObjectId('6ae82c78d4538175be752b14'),
    title: '读取偏好测试数据',
    price: 100
  }
]
```

图 6-5 从节点优先（secondaryPreferred）读取验证

```
rs0 [direct: primary] test> rs.remove("192.168.121.130:27017")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793599449, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('b+4Aun/Z3n0wSWfOsN8Yq1DPjA8=', 0),
      keyId: Long('7703445207437541382')
    }
  },
  operationTime: Timestamp({ t: 1793599449, i: 1 })
}
rs0 [direct: primary] test> rs.status().members.map(m => ({ name: m.name, state: m.stateStr
 }))
[
  { name: '192.168.121.134:27017', state: 'SECONDARY' },
  { name: '192.168.121.129:27017', state: 'PRIMARY' }
]
rs0 [direct: primary] test>
```

图 6-6 副本集节点移除演练

```
rs0 [direct: primary] test> rs.add("192.168.121.130:27017")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793599503, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('NNl77q/NciVswuAuaNpTopH4ORs=', 0),
      keyId: Long('7703445207437541382')
    }
  },
  operationTime: Timestamp({ t: 1793599503, i: 1 })
}
rs0 [direct: primary] test> rs.status().members.map(m => ({ name: m.name, state: m.stateStr
 }))
[
  { name: '192.168.121.134:27017', state: 'SECONDARY' },
  { name: '192.168.121.129:27017', state: 'PRIMARY' },
  { name: '192.168.121.130:27017', state: 'SECONDARY' }
]
rs0 [direct: primary] test>
```

图 6-7 副本集节点添加演练

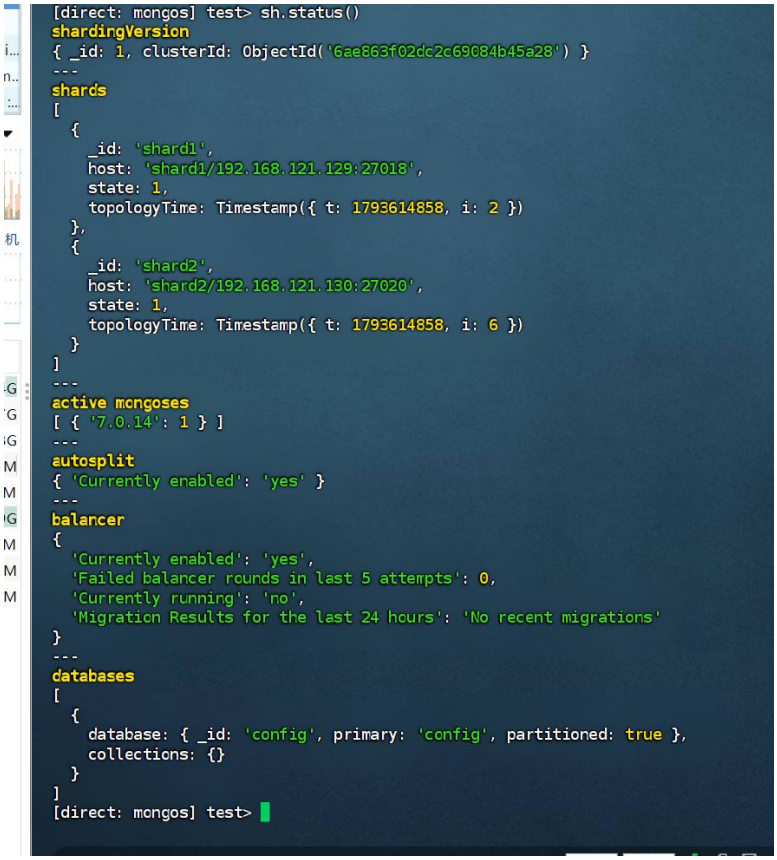


图 6-9：分片集群运行状态与分片列表

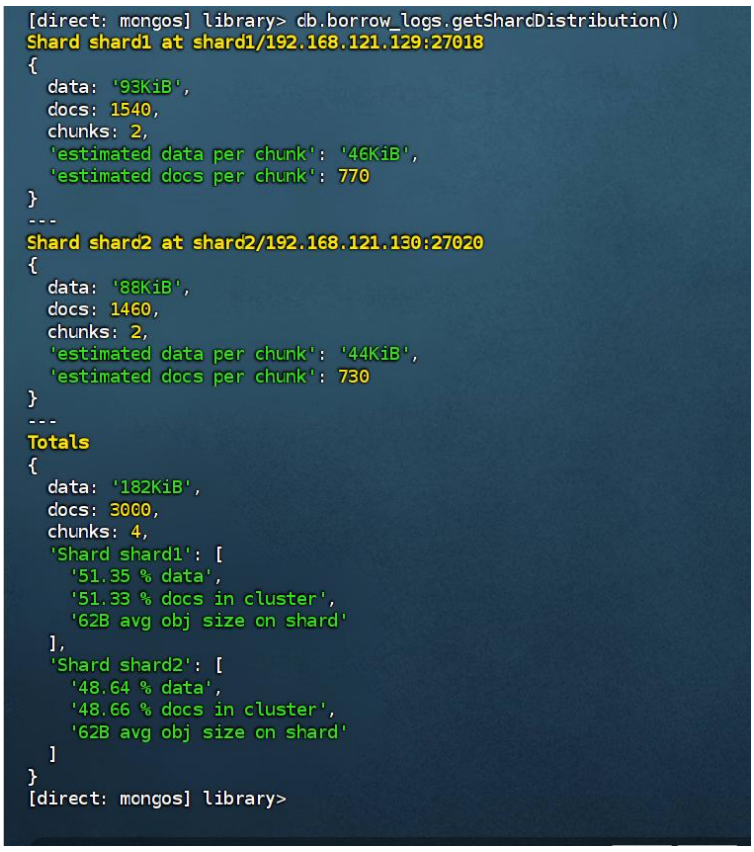


图 6-10：数据在各分片上的 chunk 及百分比分布

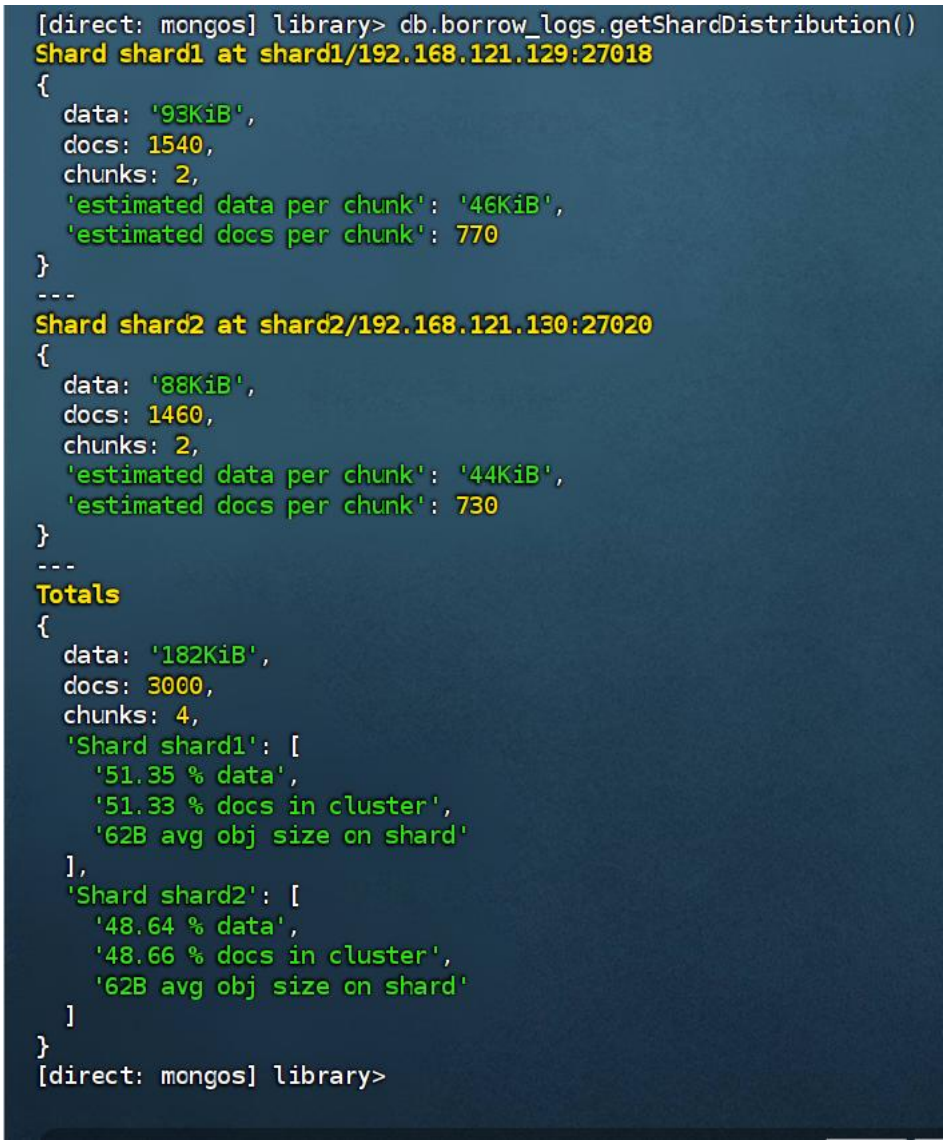


图 6-10 分片集群数据分布验证

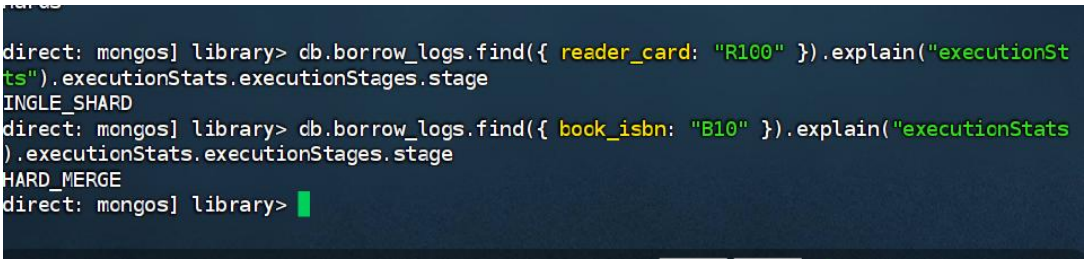


图 6-11 跨分片查询与定向查询性能对比

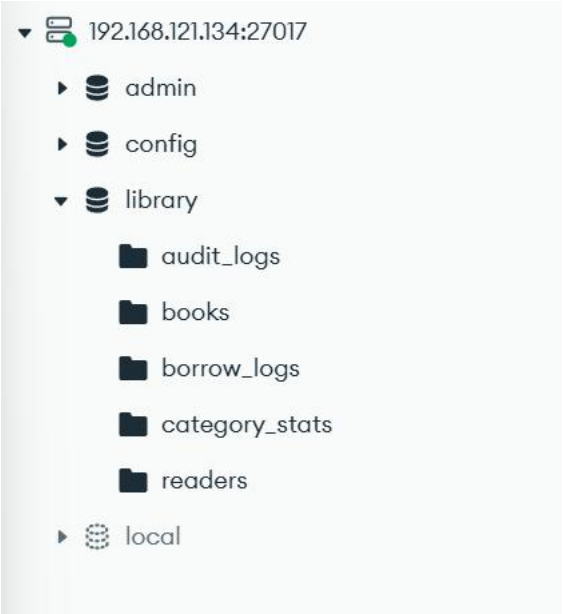

192.168.121.134:27017
admin
config
library
audit_logs
books
borrow_logs
category_stats
readers
local

图 7-1

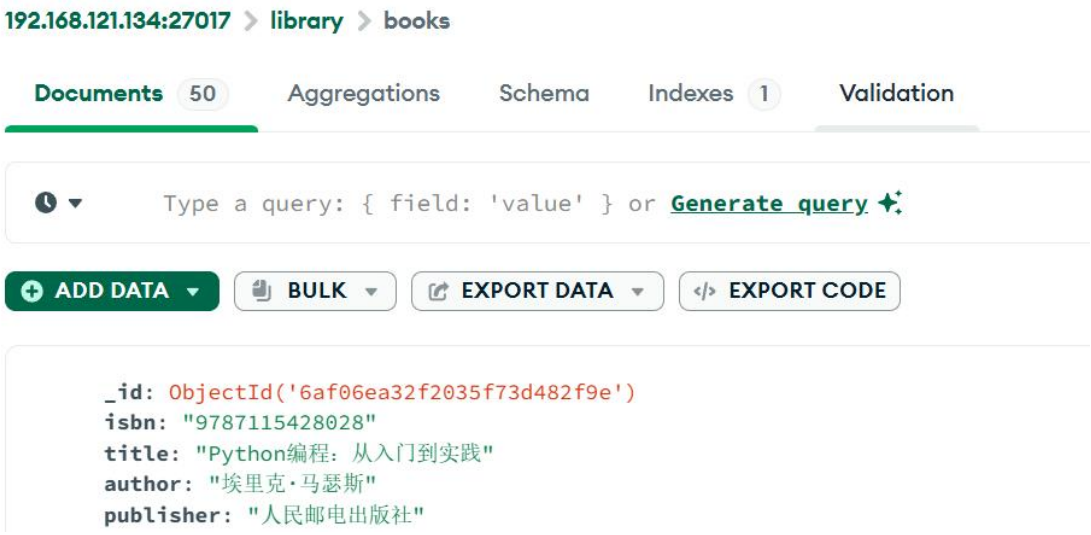

192.168.121.134:27017 > library > books
Documents 50
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
_id: ObjectId('6af06ea32f2035f73d482f9e')
isbn: "9787115428028"
title: "Python编程：从入门到实践"
author: "埃里克·马瑟斯"
publisher: "人民邮电出版社"

图 7-2

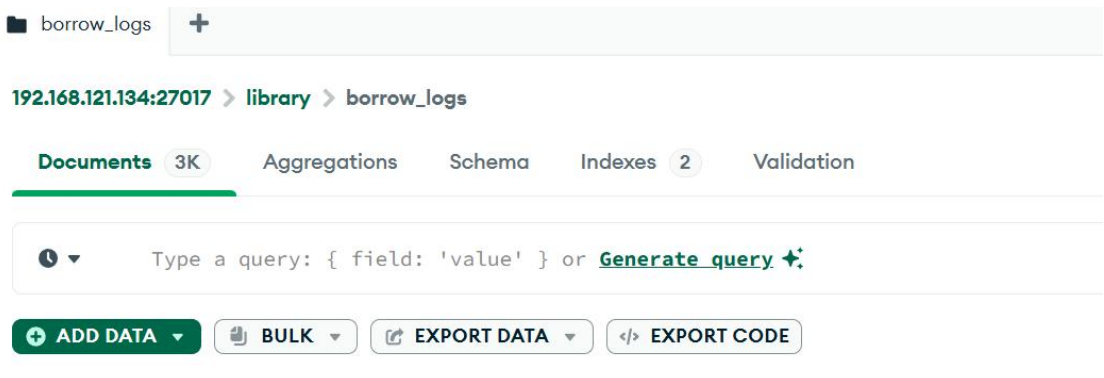

borrow_logs
192.168.121.134:27017 > library > borrow_logs
Documents 3K
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
ADD DATA
BULK
EXPORT DATA
EXPORT CODE

图 7-3

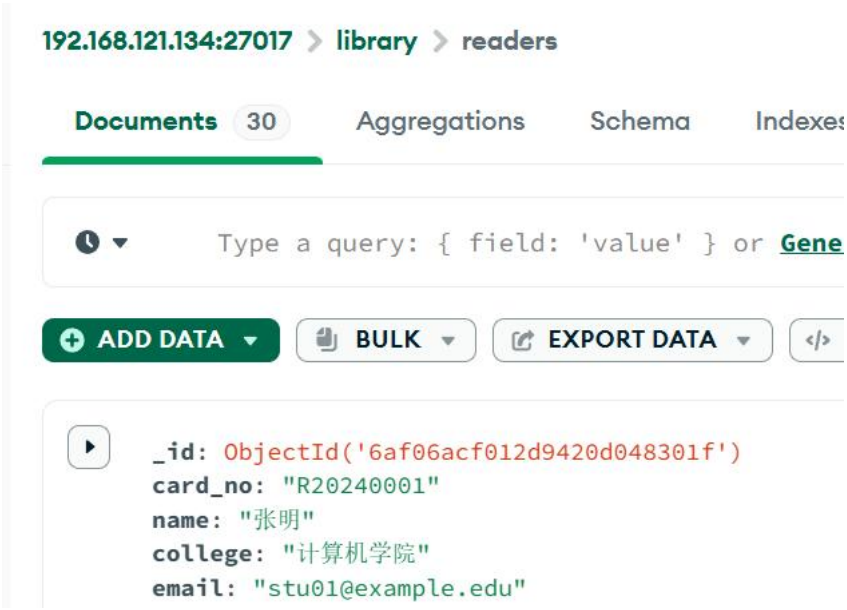


图 7-4

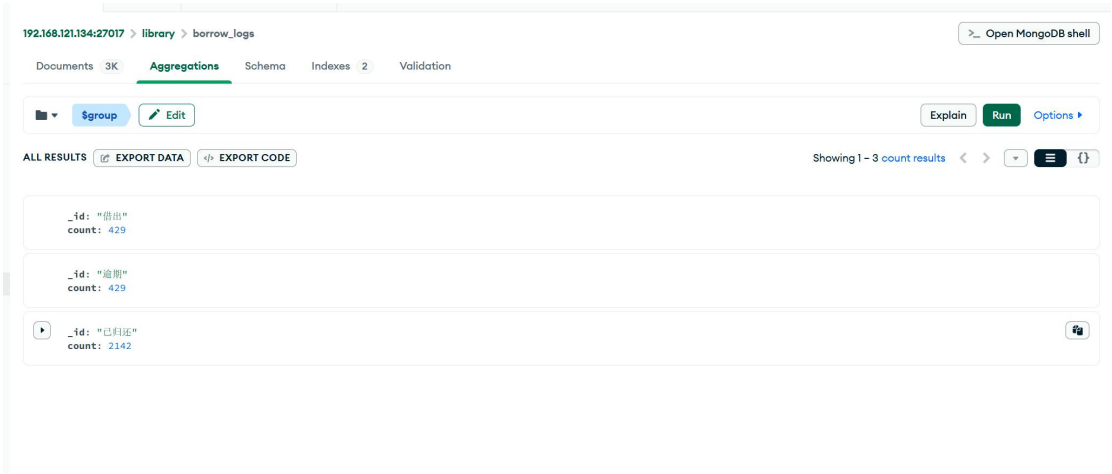


图 7-5

```
[direct: mongos] library> db.borrow_logs.aggregate([ { $group: { _id: "$status", count: { $
sum: 1 } } } ])
[
  { _id: '已归还', count: 2142 },
  { _id: '借出', count: 429 },
  { _id: '逾期', count: 429 }
]
```

图 7-6

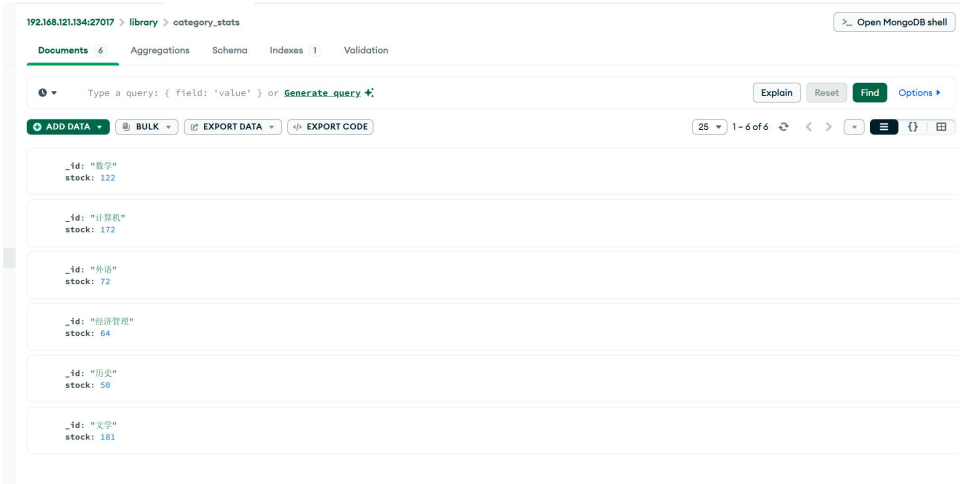


图 7-7

```
C:\Users\Tong3>ping 192.168.121.134

正在 Ping 192.168.121.134 具有 32 字节的数据:
来自 192.168.121.134 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.121.134 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.121.134 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.121.134 的回复: 字节=32 时间<1ms TTL=64

192.168.121.134 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 1ms, 平均 = 0ms

C:\Users\Tong3>
C:\Users\Tong3>ping 192.168.121.129

正在 Ping 192.168.121.129 具有 32 字节的数据:
来自 192.168.121.129 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.121.129 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.121.129 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.121.129 的回复: 字节=32 时间=1ms TTL=64

192.168.121.129 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 1ms, 平均 = 0ms

C:\Users\Tong3>
C:\Users\Tong3>ping 192.168.121.130

正在 Ping 192.168.121.130 具有 32 字节的数据:
来自 192.168.121.130 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.121.130 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.121.130 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.121.130 的回复: 字节=32 时间<1ms TTL=64

192.168.121.130 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 1ms, 平均 = 0ms
```

霍钰彤的 ping 测试

```
C:\Users\HP>ping 192.168.121.134

正在 Ping 192.168.121.134 具有 32 字节的数据：
来自 192.168.121.134 的回复：字节=32 时间=2ms TTL=64
来自 192.168.121.134 的回复：字节=32 时间=1ms TTL=64
来自 192.168.121.134 的回复：字节=32 时间=1ms TTL=64
来自 192.168.121.134 的回复：字节=32 时间=1ms TTL=64

192.168.121.134 的 Ping 统计信息：
    数据包：已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)，
往返行程的估计时间(以毫秒为单位)：
    最短 = 1ms，最长 = 2ms，平均 = 1ms

C:\Users\HP>ping 192.168.121.135

正在 Ping 192.168.121.135 具有 32 字节的数据：
来自 192.168.121.135 的回复：字节=32 时间=5ms TTL=64
来自 192.168.121.135 的回复：字节=32 时间=1ms TTL=64
来自 192.168.121.135 的回复：字节=32 时间=1ms TTL=64
来自 192.168.121.135 的回复：字节=32 时间<1ms TTL=64

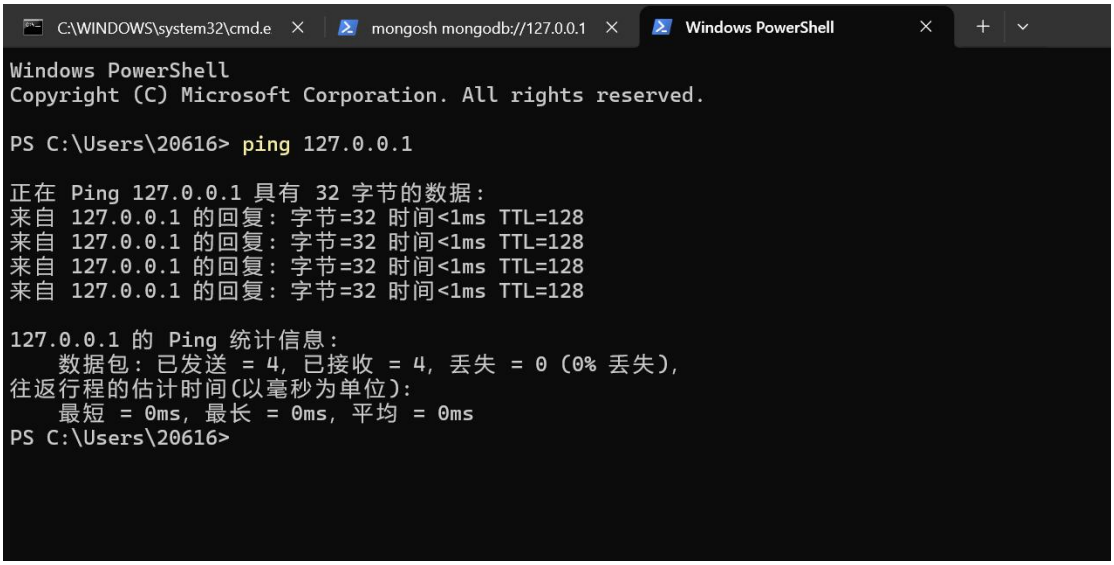
192.168.121.135 的 Ping 统计信息：
    数据包：已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)，
往返行程的估计时间(以毫秒为单位)：
    最短 = 0ms，最长 = 5ms，平均 = 1ms

C:\Users\HP>ping 192.168.121.136

正在 Ping 192.168.121.136 具有 32 字节的数据：
来自 192.168.121.136 的回复：字节=32 时间=15ms TTL=64
来自 192.168.121.136 的回复：字节=32 时间=1ms TTL=64
来自 192.168.121.136 的回复：字节=32 时间=1ms TTL=64
来自 192.168.121.136 的回复：字节=32 时间=1ms TTL=64

192.168.121.136 的 Ping 统计信息：
    数据包：已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)，
往返行程的估计时间(以毫秒为单位)：
    最短 = 1ms，最长 = 15ms，平均 = 4ms
```

杨爽茜的 ping 测试



李晨洁的 ping 测试